ΟΙ ΕΞΑΙΡΕΣΕΙΣ - Exceptions

ΣΤΗΝ PYTHON

Μ.Ι.Σ. 2025-2026

Σε κάθε γλώσσα προγραμματισμού και κατά επέκταση σε κάθε compiler (ή interpreter) μιας γλώσσας προγραμματισμού δίδεται η δυνατότητα να οριστούν με «κάποιο χειριστικό τρόπο» διαδικασίες οι οποίες θα ενεργοποιούνται αυτόματα όταν θα συμβαίνουν κατά την διάρκεια «ζωής» (εκτέλεσης) ενός προγράμματος, απρόσμενες καταστάσεις.

Οι καταστάσεις αυτές είναι στην πραγματικότητα, σοβαρά λάθη τα οποία μια γλώσσα αναγνωρίζει, και όταν συμβούν, η γλώσσα δεν μπορεί να τα χειριστεί και προκαλούν την διακοπή της ομαλής ροής του προγράμματος.

Οι καταστάσεις αυτές είναι γνωστές σε όλες τις γλώσσες προγραμματισμού και οι περισσότεροι κατασκευαστές γλωσσών προγραμματισμού παρέχουν <u>ειδικές διαδικασίες</u> ώστε την στιγμή που προκαλούνται αυτά τα λάθη, να μην διακόπτεται η λειτουργία του προγράμματος αλλά να δίνεται η δυνατότητα (να επανα-δίνεται ο έλεγχος της ροής, της «μπίλιας» του προγράμματος) στους προγραμματιστές, ώστε να τα εντοπίζουν και να μπορούν να τα διαχειρίζονται.

Η διαδικασία παρουσιάζεται (ανά κατασκευαστή και εποχή) σαν,

- ΧΕΙΡΙΣΜΟΣ ΕΞΑΙΡΕΣΕΩΝ

- Exceptions handling
- Exceptions

- ΕΞΑΙΡΕΣΕΙΣ
- Declarative handling

- ΔΙΑΚΟΠΗ
- ΧΕΙΡΙΣΜΟΣ ΔΙΑΚΟΠΩΝ

Μ.Ι.Σ. 2025-2026

Ένα παράδειγμα μιας εξαίρεσης θα μπορούσε να θεωρηθεί η «αλίευση» της απότομης διακοπής της ομαλής ροής ενός προγράμματος εάν προσπαθήσουμε για παράδειγμα να διαιρέσουμε με το μηδέν. Το αποτέλεσμα από μια τέτοια κατάσταση θα ήταν να σταματήσει το πρόγραμμα και να διακοπεί η λειτουργία της εφαρμογής.

Η εγκατάσταση μιας εξαίρεσης είναι η διαδικασία με την οποία δεν θα επιτραπεί να διακοπεί η λειτουργία του προγράμματος (καθώς αυτό υποστηρίζει την λειτουργία των χειρουργείων ενός νοσοκομείου) αλλά να εμφανιστεί ένα απλό προειδοποιητικό μήνυμα, (για παράδειγμα, «***παρακαλούμε καλέστε το τμήμα μηχανογράφησης και αναφέρετε τον κωδικό A786BB0***») και το πρόγραμμα του νοσοκομείου θα συνεχίζει να λειτουργεί.

Παρόμοιες καταστάσεις (όπως η ανωτέρω περιγραφόμενη) μπορεί να είναι η προφύλαξη από την έξοδο εκτός των ορίων ενός πίνακα, η αναζήτηση ενός ανύπαρκτου record, κ.λπ.

Σε όλες τις γλώσσες προγραμματισμού οι καταστάσεις αυτές, που μπορούμε να διαχειριστούμε είναι πολλές και σύνθετες και θα πρέπει να αναζητούνται σχετικές πληροφορίες στα εγχειρίδια του κατασκευαστή της γλώσσας.
Αυτή η αναζήτηση είναι σημαντική, καθώς η σύνταξη μιας ΕΞΑΙΡΕΣΗΣ και η ορισμός του ΧΕΙΡΙΣΜΟΥ ΕΞΑΙΡΕΣΗΣ, μέσα σε ένα πρόγραμμα, διαφέρει σημαντικά για κάθε compiler (ή interpreter) και φυσικά για κάθε γλώσσα προγραμματισμού.

Η Python διαθέτει μηχανισμούς για τον χειρισμό αυτών των καταστάσεων.

Για παράδειγμα, εάν προσπαθήσουμε να διαβάσουμε ένα αρχείο το οποίο δεν υπάρχει στον σκληρό μας δίσκο, θα λάβουμε ένα μήνυμα λάθους, αλλά η «εγκατάσταση» μιας εξαίρεσης θα μας προφυλάξει από αυτή την απότομη διακοπή που θα συνέβαινε σε άλλη περίπτωση.

Μ.Κ. 2025-2026

Για παράδειγμα, εάν θέλουμε να ορίσουμε μια τέτοια συμπεριφορά θα πρέπει να γράψουμε,

```
my_file = input ('WHAT IS THE NAME OF YOUR DISK FILE…')

try:
        my_check = open (my_file, 'r')

        print (my_check.read())

        my_check.close()

except        FileNotFoundError :

              print ('THIS FILE DOES NOT EXISTS!')
```

στην ανωτέρω περίπτωση η γλώσσα θα απαντήσει με κατάλληλο μήνυμα για την περίπτωση που δοθεί ανύπαρκτο αρχείο.

Μ.Ι.Σ. 2025-2026

Στην Python η ορθή σύνταξη γίνεται σε δύο κομμάτια,

στο πρώτο

με το **try:**
ορίζουμε την/τις εντολή/ές που θα διαθέτει/τουν, τον έλεγχο ΕΞΑΙΡΕΣΗΣ

στο δεύτερο
με το **except**
ορίζουμε για κάθε ένα ή για πολλά (απρόσμενα) γεγονότα
FileNotFoundError:
που εάν συμβούν
τότε στην περίπτωση αυτή, θα εκτελεστούν οι εντολές
που γράφουμε κάτω από το συγκεκριμένο απρόσμενο γεγονός

Η Python διαθέτει μεγάλη ποικιλία λόγων προφύλαξης από απρόσμενα γεγονότα, όπως τα,

ValueError:
ZeroDivisionError:
IndexError:
TypeError:
EOFError:

Μ.Ι.Σ. 2025-2026

Για παράδειγμα, μπορούμε να γράψουμε κώδικα ο οποίος θα ελέγχει ατέρμονα εάν εισάγουμε μια ορθή τιμή,

```
.
.
.
def  my_check_routina():

        while True:

                try:
                        my_accept_from_keyboard = int (input('GIVE AN INTEGER NUMBER…'))

                        return  my_accept_from_keyboard

                except        ValueError:

                                print ('ONLY INTEGER NUMBERS!')

.
.
.
yes_this_is_my_int_number = my_check_routina()
print(yes_this_is_my_int_number)
.
.
.
```

στην ανωτέρω περίπτωση η γλώσσα θα συνεχίζει ατέρμονα να ζητά πληκτρολόγηση μέχρι να λάβει έναν ακέραιο αριθμό.

Μ.Ι.Σ. 2025-2026

Καθώς η Python διαθέτει πολλές περιπτώσεις απρόσμενων γεγονότων, μπορούμε να παρατηρήσουμε αυτά τα γεγονότα από το μήνυμα λάθους που θα λάβουμε όταν η γλώσσα διακόψει με απότομο μήνυμα την ροή του προγράμματος.

Έτσι να επανέλθουμε στο κατάλληλο σημείο ώστε να εισάγουμε το κατάλληλο κώδικα ελέγχου της συγκεκριμένης διακοπής.

Αυτά τα μηνύματα λάθους της γλώσσας Python εμφανίζονται αρχικά με το **NameError:** του προβλήματος που προέκυψε καθώς και <u>το σημείο</u> που προκλήθηκε το απρόσμενο πρόβλημα.

Αυτό επιτρέπει στον προγραμματιστή να αναγνωρίσει το λάθος του να βρει το **NameError:** του προβλήματος που προκαλείται και έτσι <u>στο σημείο αυτό</u> να εισάγει σχετικό κώδικα με το κατάλληλο **NameError:** κάτω από το **except** και έτσι να ελέγξει αυτή την απρόσμενη κατάσταση.

Μ.Ι.Σ. 2025-2026

Επιπρόσθετα σε κάθε **try:** μπορούμε να έχουμε πολλά **except**

```
try:
        .
        .
        .

except      FileNotFoundError :
        .
        .
        .
except      ZeroDivisionError:
        .
        .
        .
except      EOFError:
        .
        .
        .
else
        .
        .
        .
```

ανάλογα με το ποια εντολή από το σώμα της **try** πάρουμε απρόσμενο λάθος θα ενεργοποιηθεί αυτόματα ο αντίστοιχος μηχανισμός ελέγχου της διακοπής.

Μ.Ι.Σ. 2025-2026

Η εντολή **except** μπορεί να χειριστεί ένα απρόσμενο λάθος ή μια εξαίρεση, αλλά και μια λίστα σφαλμάτων/εξαιρέσεων μέσα σε παρένθεση.

except(RuntimeError, TypeError, NameError):

Επιπλέον εάν δεν τοποθετήσουμε ονομασίες σφαλμάτων ή εξαιρέσεων, μια σκέτη **except** θα χειριστεί όλες τις εξαιρέσεις και όλα τα σφάλματα (οτιδήποτε απρόσμενο λάθος συμβεί).

Επιπλέον μπορούμε να έχουμε και ένα **else** με κατάλληλες εντολές. Αυτή η **else,** η οποία βέβαια αφορά το σώμα των εντολών που διαθέτει **try** θα εκτελεστεί εάν δεν συμβαίνει καμία εξαίρεση, από αυτές που έχουν δηλωθεί και όλα πάνε καλά.

Μ.Ι.Σ. 2025-2026

Η Python διαθέτει και μια πρόσθετη ενδιαφέρουσα σύνθεση στην δήλωση των εξαιρέσεων, συγκεκριμένα,

```
try:
        .
        .
        .

except      FileNotFoundError :
        .
        .
        .
except      ZeroDivisionError:
        .
        .
        .
finally:
        print ('CALL THE POLICE!')
```

στην ανωτέρω σύνθεση, εάν ενεργοποιηθεί κάποια **except** θα εκτελεστεί το σύνολο των εντολών της και θα ακολουθήσει αυτόματα και το **finally:** με όσες, αυτό, πρόσθετες εντολές περιέχει.

M.I.Σ. 2025-2026